

**Riphah International University Gulberg Green Campus
Islamabad
Faculty of Computing**



Artificial Intelligence

Project Report

***The Closet Fashion Assistant
(AI-Based Smart Outfit Recommendation System)***

Submitted By:

Name	Program	Semester	SAP ID
Afifa Amna	BS-SE	6 th	55466
Anosha Pervaiz Awan	BS-SE	6 th	56200
Areeza Afridi	BS-SE	6 th	55254
Nighat Naseem	BS-SE	6 th	53740
Nimra Tariq	BS-SE	6 th	54909
Simaab Malik	BS-SE	6 th	54910

**Submitted to: Ma'am Shumaila Qayyum
Date of Submission: 09/3/2026**

Table of Contents

1. Introduction	3
2. ML Technique Selection.....	3
2.1 Chosen Technique: K-Nearest Neighbors (KNN)	3
2.2 Secondary Technique: Naive Bayes (for Occasion Classification)	3
3. Dataset Description	3
3.1 Dataset Overview	3
3.2 Feature Description.....	4
3.3 Dataset Generation Methodology.....	5
3.5 Dataset File: closet_fashion_dataset.csv	5
4. Data Preprocessing	5
4.1 Encoding Categorical Features	5
4.2 Feature Normalization.....	5
4.3 Train-Test Split	5
5. KNN Model Training.....	6
5.1 Model Configuration.....	6
5.2 Training Code (Python / scikit-learn)	6
5.3 K-Value Optimization.....	7
6. Evaluation Metrics	7
6.1 Target Performance Thresholds	7
7. Naive Bayes — Occasion Classifier	7
7.1 Purpose	7
7.2 Training Approach.....	8
8. Integration with Agent Pipeline	8
9. Summary	9

1. Introduction

The complete Machine Learning Component Plan for The Closet Fashion Assistant is presented in this document. The document explains which ML technique was chosen and why this specific ML approach was selected while providing information about the dataset and training and testing methods and evaluation metrics which show that the system uses complete documentation to implement one ML technique.

The system uses K-Nearest Neighbors (KNN) as its primary ML technique. The utility function which operates the recommendation engine uses KNN to calculate style similarity scores between candidate outfits and the user's previously approved outfits.

2. ML Technique Selection

2.1 Chosen Technique: K-Nearest Neighbors (KNN)

KNN serves as the main machine learning algorithm because of its three advantages which are:

- **Interpretability** — KNN decisions are explainable by showing 'you liked similar outfits before', which is critical for a user-facing recommendation app.
- **No retraining from scratch** — KNN system requires no full system retraining because it functions as an instance-based system which enables instant model advancement through the addition of new preferred outfits.
- **Handles multi-type features well** — KNN algorithm can effectively process multiple types of features because it transforms outfit characteristics such as color and pattern and occasion into numeric vector formats required for distance calculations.
- **Lightweight for mobile** — KNN systems with less than 500 wardrobe items operate on mobile devices within milliseconds.
- **Proven in recommendation systems** — KNN functions as the standard benchmark for recommendation systems which use both collaborative and content-based filtering methods.

2.2 Secondary Technique: Naive Bayes (for Occasion Classification)

The system uses a Naive Bayes classifier as an additional component which determines the appropriate occasion category for an outfit based on its characteristics. The system now includes probabilistic reasoning through this development.

3. Dataset Description

3.1 Dataset Overview

The research project developed a structured dataset which researchers named **closet_fashion_dataset.csv**. The dataset includes 500 outfit item records which contain 17 attributes each for simulating actual wardrobe situations that include various clothing types and occasions and weather conditions and user acceptance patterns.

Property	Value	Details
Total Records	500	Outfit item entries
Features	17	Mix of categorical and numerical
Target Variable	outfit_accepted	Binary (0 = rejected, 1 = accepted)
Positive Class (1)	~58%	Accepted outfits
Negative Class (0)	~42%	Rejected outfits
File Format	CSV	closet_fashion_dataset.csv

3.2 Feature Description

The table below describes all 17 columns in the dataset:

Column Name	Type	Values / Range	Description
item_id	Integer	1 – 500	Unique identifier for each clothing item
category	Categorical	top, bottom, shoes	Type of clothing item
color	Categorical	10 colors	Color of the clothing item
pattern	Categorical	solid, striped, etc.	Visual pattern of the item
material	Categorical	cotton, denim, wool...	Fabric/material type
occasion_type	Categorical	casual, formal, wedding...	Intended occasion
season_suitability	Categorical	summer, winter, both	Seasonal fit of the item
weather_condition	Categorical	sunny, rainy, cold...	Current weather context
temperature_C	Float	5.0 – 45.0	Ambient temperature in Celsius
wear_count_last30days	Integer	0 – 15	How often worn in last 30 days
days_since_last_worn	Integer	0 – 60	Recency of last wear
color_harmony_score	Float	0.30 – 1.00	Color matching quality (0–1)
occasion_match_score	Float	0.20 – 1.00	Occasion suitability score (0–1)
weather_match_score	Float	0.20 – 1.00	Weather suitability score (0–1)
diversity_score	Float	0.00 – 1.00	Variety score (avoids repetition)
utility_score	Float	Computed	Weighted utility: $0.35W + 0.30O + 0.20C + 0.15D$
outfit_accepted	Binary	0 or 1	Target label — user acceptance

3.3 Dataset Generation Methodology

The Closet Fashion Assistant currently lacks actual user data which led to the creation of its dataset through synthetic data generation using an established simulation method. The simulation data was created through the following methods:

- The system generates random samples of categorical attributes which represent actual domain values that include colors and materials and occasions.
- The simulation employ Uniform (5, 45) temperature distribution to achieve complete seasonal temperature range simulation.
- The domain rules generated scores which used **weather_match_score** calculation based on temperature and condition mapping.
- The project's established formula calculates utility score through **$0.35 \times \text{Weather} + 0.30 \times \text{Occasion} + 0.20 \times \text{Color} + 0.15 \times \text{Diversity}$** .
- The system generates target label (**outfit_accepted**) through probabilistic methods: **outfits with utility > 0.55 receive 85% acceptance probability** while those below **0.55 receive 10% acceptance probability** which simulates realistic user behavior with some noise.

The approach creates a dataset which maintains class balance and authenticates the system's true decision-making procedures.

3.5 Dataset File: closet_fashion_dataset.csv

4. Data Preprocessing

4.1 Encoding Categorical Features

The process of KNN implementation begins with One-Hot Encoding of categorical columns which include category, color, pattern, material, occasion_type, season_suitability, and weather_condition through the utilization of pandas get_dummies() and sklearn LabelEncoder.

4.2 Feature Normalization

The scaling of all numerical features to the range of [0, 1] will be achieved through Min-Max Normalization which prevents KNN distance computations from being influenced by features that have large numeric ranges such as temperature_C.

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X_train)
```

4.3 Train-Test Split

The dataset will be split 80/20: 400 records for training and 100 records for testing. Stratified splitting ensures both classes are proportionally represented in each split.

5. KNN Model Training

5.1 Model Configuration

The KNN classifier will be configured as follows:

Parameter	Value & Justification
k (neighbors)	k = 5 — standard default; validated via cross-validation
Distance Metric	Euclidean — suitable for normalized continuous features
Weighting	Uniform — all 5 neighbors weighted equally
Algorithm	Ball Tree — efficient for medium-sized datasets
Feature Input	One-hot encoded + normalized numeric features
Target	outfit_accepted (binary classification)

5.2 Training Code (Python / scikit-learn)

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# Load dataset
df = pd.read_csv('closet_fashion_dataset.csv')

# One-hot encode categorical columns
cat_cols = ['category', 'color', 'pattern', 'material',
            'occasion_type', 'season_suitability', 'weather_condition']
df_encoded = pd.get_dummies(df, columns=cat_cols)

# Define features and target
X = df_encoded.drop(columns=['item_id', 'outfit_accepted'])
y = df_encoded['outfit_accepted']

# Normalize numerical features
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

# Train/test split (80/20, stratified)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y)

# Train KNN model
knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
knn.fit(X_train, y_train)

# Evaluate
y_pred = knn.predict(X_test)
print('Accuracy:', accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
print('Confusion Matrix:')
print(confusion_matrix(y_test, y_pred))
```

5.3 K-Value Optimization

The optimal k value will be determined through cross-validation for the k values which include 1 3 5 7 9 11. The k value that achieves the greatest mean accuracy will determine the final model parameters.

6. Evaluation Metrics

The following metrics will be used to evaluate model performance, consistent with the project proposal's Evaluation Metrics section:

Metric	Formula	Relevance to System
Accuracy	$TP+TN / \text{Total}$	Overall correctness of outfit accept/reject predictions
Precision	$TP / (TP+FP)$	Of outfits predicted as accepted, how many were truly good? Avoids bad suggestions.
Recall	$TP / (TP+FN)$	Of truly good outfits, how many were recommended? Avoids missing good options.
F1-Score	$2 \times P \times R / (P+R)$	Harmonic mean — balances precision and recall for imbalanced data.
Confusion Matrix	2×2 matrix	Visualizes TP, TN, FP, FN breakdown for full error analysis.
Precision@3	Accepted among top 3 shown	Directly measures user experience — are the top recommendations good?
Cross-Val Score	Mean $\pm$ Std across 5 folds	Ensures model is not overfitting to the training split.

6.1 Target Performance Thresholds

- Accuracy $\geq 80\%$
- F1-Score ≥ 0.78
- Precision@3 ≥ 0.70 (at least 2 of top 3 recommendations accepted by user)
- Repetition Rate < 10% across 30 days of recommendations

7. Naive Bayes — Occasion Classifier

7.1 Purpose

The primary machine learning system uses numerical features of outfits which include temperature and wear counts and scores to create a Gaussian Naive Bayes classifier that predicts the best occasion type for an outfit. The system generates a probability distribution which shows the different occasion types that feeds into the function used to calculate utility scores.

7.2 Training Approach

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Use numerical features only for Naive Bayes
num_features = ['temperature_C', 'wear_count_last30days', 'days_since_last_worn',
                'color_harmony_score', 'occasion_match_score',
                'weather_match_score', 'diversity_score', 'utility_score']

X_nb = df[num_features]
y_nb = df['occasion_type'] # multi-class target

X_train_nb, X_test_nb, y_train_nb, y_test_nb = train_test_split(
    X_nb, y_nb, test_size=0.2, random_state=42, stratify=y_nb)

nb = GaussianNB()
nb.fit(X_train_nb, y_train_nb)
y_pred_nb = nb.predict(X_test_nb)
print('NB Accuracy:', accuracy_score(y_test_nb, y_pred_nb))
```

8. Integration with Agent Pipeline

Both trained models integrate into the agent's recommendation pipeline as follows:

Step	Component	ML Role
1	Filtering	Rule-based (no ML) — removes dirty clothes and season-incompatible items
2	Candidate Generation	Rule-based Cartesian product of filtered items
3	KNN Similarity	Trained KNN computes style_similarity_score for each candidate outfit vs. liked history
4	NB Occasion Scoring	Trained NB gives occasion probability; feeds into occasion_match_score
5	Utility Calculation	Scores combined: $0.35W + 0.30O_NB + 0.20KNN_style + 0.15D$
6	A* Search	Selects outfit with max utility score
7	Feedback Loop	User accept/reject updates KNN training set; model retrained every 10 interactions

9. Summary

The ML Plan achieves its goal through one machine learning method which needs three elements: a dataset and training/testing data and evaluation metrics. The following elements demonstrate this requirement:

- The **closet_fashion_dataset.csv dataset** contains 500 records which include 17 features and one binary target (outfit_accepted)
- **Machine Learning Technique 1 — KNN (k=5):** The algorithm learned to predict outfit acceptance and assess style similarity The evaluation used accuracy precision recall F1 score confusion matrix and 5-fold cross validation results
- **Machine Learning Technique 2 — Naive Bayes:** The algorithm learned to classify occasions based on numerical outfit attributes
- The dataset underwent a stratified **80/20 split process** which used **MinMax** normalization together with one hot encoding
- The system needs to achieve **80 accuracy** and **0.78 F1 score** and **0.70 Precision@3 score**

The two machine learning components function as a single unit within the **Utility Based Agent System** which follows the design specifications of the **Algorithm Design Plan** to deliver intelligent recommendation services.